



Code Convention

- 팀 내 일관된 코드 스타일과 품질을 유지하기 위한 코드 컨벤션

코드 스타일 가이드

네이밍 규칙

대상	규칙	예시
클래스명	PascalCase	MemberController , MemberService
변수명	camelCase	menuName , memberId
메서드명	camelCase	createMember() , getMemberList()
패키지명	소문자	entity , service , controller
상수명	UPPER_SNAKE_CASE	MAX_SIZE , DEFAULT_PAGE_SIZE

코드 스타일

- 구글 스타일 가이드 적용
- 들여쓰기: 스페이스 4칸
- 띄어쓰기: Formatter 자동 적용
- 줄 길이: 120자 제한

Google Style Guide 적용

- 일관된 띄어쓰기와 들여쓰기로 코드 스타일 통일
- 들여쓰기: 스페이스 4칸
- 줄 길이: 120자 제한
- 구글 시트 공유를 통한 줄맞춤 자동 설정

커스텀 1 : 2 1 2 2

커스텀 2 : 4 4 4 0

커스텀 3 : 100 → 140

스타일 가이드 실행 단축키 : `⌘ L`

→ 필요한 클래스 내에서 단축키 `Option + Command + L` 을 누르면 자동으로 적용

<https://velog.io/@tkddnwkdb/IntelliJ-Google-Style-Guide-%EC%A0%81%EC%9A%A9>

📁 패키지 구조 (DDD 방식)

도메인별 패키지 분리

```
src/main/java/com/project/
├── member/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   ├── entity/
│   └── dto/
├── payment/
│   ├── controller/
│   ├── service/
│   └── ...
├── common/
└── response/
```

|— exception/
|— config/

아키텍처 및 패턴

빌더 패턴 사용

```
// BuilderPattern 사용
@Builder
Member member = Member.builder()
    .name("김영희")
    .email("kimyoung@example.com")
    .age(27)
    .build();
```

DTO 생성자 명시적 작성

```
// RequestDto @Builder → @NoArgsConstructor
@Getter
@NoArgsConstructor

// DTO 생성자 명시적 작성
public class MemberResponseDto {
    private String name;
    private String email;

    // Entity를 받는 생성자
    public MemberResponseDto(Member member) {
        this.name = member.getName();
        this.email = member.getEmail();
    }
}
```

데이터 전달 방식

Controller ↔ Service 데이터 전달

```

// Controller → Service: RequestDto 풀어서 전달
@PostMapping("/members")
public ResponseEntity<CommonResponseDto<MemberResponseDto>> createMember(
    @RequestBody MemberRequestDto requestDto) {
    // requestDto, responseDto 의 이름이 너무 길 경우 앞에 생략하고
    // requestDto, responseDto로 반환

    // RequestDto를 풀어서 Service에 전달
    MemberResponseDto responseDto = memberService.createMember(
        requestDto.getName(),
        requestDto.getEmail(),
        requestDto.getAge()
    );

    return ResponseEntity.ok(new CommonResponseDto<>("회원 생성 완료",
        responseDto));
}

// Service → Controller: ResponseDto 반환
public MemberResponseDto createMember(String name, String email, int age) {
    Member member = Member.builder()
        .name(name)
        .email(email)
        .age(age)
        .build();

    User savedMember = memberRepository.save(member);
    return new MemberResponseDto(savedMember);
}

```

예외 상황

- RequestDto 내부 값이 3개 이상인 경우: Dto 전체를 전달 가능



객체 변환 방식

RequestDto → Entity 변환

```
// RequestDto → Entity : Spread 방식 사용
public Member createMember(String name, String email, int age) {
    return Member.builder()
        .name(name)
        .email(email)
        .age(age)
        .build();
}
```

Entity → ResponseDto 변환

```
// Entity → ResponseDto : Constructor 방식 사용
public class MemberResponseDto {
    public MemberResponseDto(Member member) {
        this.name = member.getName();
        this.email = member.getEmail();
    }
}
```



공통 응답 형식

CommonResponseDto<T> 사용

```
// 단일 객체 응답
public class CommonResponseDto<T> {
    private String message;
    private String status;
    private T data;

    public CommonResponseDto(String message, String status, T data) {
        this.message = message;
        this.status = status;
        this.data = data;
    }
}
```

```
// 리스트 응답
public class CommonListResponseDto<T> {
    private String message;
    private String status;
    private List<T> data;

    public CommonListResponseDto(String message, String status, List<T>
data) {
        this.message = message;
        this.status = status;
        this.data = data;
    }
}
```

Controller에서 ResponseEntity 사용

```
@PostMapping("/members")
public ResponseEntity<CommonResponseDto<MemberResponseDto>> cr
eateMember(
    @RequestBody MemberRequestDto requestDto) {

    MemberResponseDto responseDto = memberService.createMember(
        requestDto.getName(),
        requestDto.getEmail()
    );

    return ResponseEntity.ok(new CommonResponseDto<>("회원 생성 완료",
responseDto));
}
```

사용 금지 사항

Annotation 사용 제한

```
// 금지
@AllArgsConstructor
```

```
@Data
```

```
// 권장
```

```
@Builder
```

```
@Getter
```

```
// RequestDto @Builder → @NoArgsConstructor  
@NoArgsConstructor
```

```
// 필요시 개별 생성자 직접 작성
```

Setter 사용 금지

```
// 금지
```

```
// member.setName("김영희");
```

```
// 권장 : 의미 있는 메서드명 사용
```

```
member.updateName("김영희");// 의미 있는 메서드명 사용
```

Getter 사용 지양

```
// 지양 (N+1 문제 가능성)
```

```
// List<Comment> comments = post.getComments();
```

```
// 권장 : Repository 직접 조회
```

```
List<Comment> comments = commentRepository.findById(postId);
```

Import 문 와일드카드 금지

```
// 금지
```

```
import java.util.*;
```

```
// 권장
```

```
import java.util.List;
```

```
import java.util.ArrayList;
```

```
import java.util.Optional;
```

Impl 사용 금지

```
// 금지
public class MemberServiceImpl implements MemberService {
}

// 권장
public class MemberService {
}
```



메서드 네이밍 규칙

HTTP 메서드별 네이밍

HTTP 메서드	Service 메서드	Repository 메서드
POST	createMember()	save()
GET	getMember()	findById()
PUT/PATCH	updateMember()	save()
DELETE	deleteMember()	deleteById()

메서드명 작성 원칙

- 축약하지 않고 명확하게 작성
- 동사 + 명사 형태로 구성
- 의미를 정확히 전달할 수 있는 이름 사용

```
// 축약된 메서드명 금지
getMer();
createMer();

// 권장 : 명확한 메서드명
getMemberById();
createMemberAccount();
```



Enum 사용 규칙

기본 형식

```
// 권장 : value 값 없이 대문자로만
public enum MemberStatus {
    ACTIVE,
    INACTIVE,
    SUSPENDED
}

// 금지 (value 값 사용 X)
public enum MemberStatus {
    ACTIVE("활성"),
    INACTIVE("비활성")
}
```

적용 범위

- 모든 상태값은 Enum 사용
- 에러 메시지도 Enum으로 관리

예외 처리

GlobalExceptionHandler 사용

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(CustomException.class)
    public ResponseEntity<CommonResponseDto<Void>> handleCustomException(
        CustomException e) {

        return ResponseEntity.status(e.getErrorCode().getHttpStatus())
            .body(new CommonResponseDto<>(e.getMessage(), null));
    }
}
```

에러 메시지 Enum 관리

```
// 모든 상태값과 에러 메시지는 Enum으로 관리
public enum ErrorCode {
    USER_NOT_FOUND(HttpStatus.NOT_FOUND, "회원을 찾을 수 없습니다."),
    INVALID_INPUT(HttpStatus.BAD_REQUEST, "잘못된 입력값입니다.");

    private final HttpStatus httpStatus;
    private final String message;
}
```



주석 작성 규칙

주석 형식

```
/**
 * 회원 정보를 생성합니다.
 * 본인 서비스에서 바로 파악하기 어려운 로직은 무조건 주석 추가
 */
public MemberResponseDto createMember(String name, String email) {
    // QueryDSL을 사용한 복잡한 쿼리 로직
    // 성능 최적화를 위해 fetch join 적용
    return memberRepository.findMemberWithDetails(name, email);
}
```

주석 작성 기준

- **Service** 로직 위에 간단하게 작성
- 새로운 기술 스택 사용 시 주석 추가
- 이해가 어려운 로직에는 반드시 주석 추가
- 리팩토링 시 주석도 함께 수정



로깅 규칙

@Slf4j 사용

```

@Slf4j
@Service
public class MemberService {

    public MemberResponseDto createMember(String name, String email) {
        log.info("회원 생성 시작: name={}, email={}", name, email);

        try {
            log.debug("회원 생성 중간 단계 완료");
            return responseDto;
        } catch (Exception e) {
            log.error("회원 생성 실패: {}", e.getMessage(), e);
            throw new CustomException(ErrorCode.USER_CREATE_FAILED);
        }
    }
}

```

제네릭 타입 규칙

제네릭 타입 명확하게 작성

```

// 와일드 카드 사용 금지
CommonResponseDto<?> response;
List<?> list;

// 명확한 타입 지정
CommonResponseDto<MemberResponseDto> response;
List<MemberResponseDto> list;

```

HTTP 파일 공유

- API 테스트용 HTTP 파일 팀 내 공유
- 엔드포인트 테스트 표준화

추가 권장 사항

검증 로직

```
// DTO 레벨에서 검증

@Getter
@NoArgsConstructor
public class MemberRequestDto {
    @NotBlank(message = "이름은 필수입니다.")
    @Size(min = 2, max = 10, message = "이름은 2-10자 사이여야 합니다.")
    private String name;

    @Email(message = "올바른 이메일 형식이 아닙니다.")
    private String email;
}
```

성능 최적화

- N+1 문제 방지를 위한 QueryDSL 활용
- 페이징 처리 시 Pageable 사용
- 캐싱 전략 고려

테스트 코드 작성

- 단위 테스트 작성 권장
- 통합 테스트로 API 검증
- 테스트 데이터 분리 관리

권한 관련 controller 로직 규칙

- 권한 제어는 @PreAuthorize 중심으로
- API 개수 기준으로 컨트롤러 분리
 - 권한별 API가 적으면(1~7개) → 하나의 Controller에 묶고 메서드 단위
`@PreAuthorize`
 - 권한별 API가 많으면(8개 이상) → 전용 Controller 분리하고 클래스 단에 공통
`@PreAuthorize`

```

@RestController
@RequestMapping("/magazines")
@PreAuthorize("hasRole('ADMIN')")
public class AdminMagazineController {

    @PostMapping
    public void createMagazine() {}

    @PutMapping("/{id}")
    public void updateMagazine() {}
}

```

SuccessCode 관리 및 기준 정리

- **CRUD 기본값 고정:** CREATE / FETCH / UPDATE / DELETE → 모든 도메인 공통 사용
- **추가 SuccessCode 생성 조건**
 - 프론트에 노출되는 메시지 필요할 때 (예: EMAIL_VERIFY_SUCCESS)
 - 도메인 흐름상 상태 구분이 중요한 경우 (예: ONBOARDING_COMPLETE_SUCCESS, SESSION_RESUME_SUCCESS)
 - 파일/AI/인증 등 CRUD에 포함되지 않는 별도 기능 처리 결과
- **불필요한 세분화 방지:** 내부 처리용 메시지는 SuccessCode로 만들지 않고 로그에서만 관리
- **네이밍 규칙:** [기능]_SUCCESS 형식으로 통일

코딩 할 때, 반드시 확인할 것!

- ☐ 네이밍 규칙 준수
- ☐ 패키지 구조 확인
- ☐ 구글 스타일 가이드 적용
- ☐ 빌더 패턴 사용

- ☐ 금지 어노테이션 확인 (`@AllArgsConstructor`)
 - ☐ DTO 생성자 직접 작성
 - ☐ Setter/Getter 사용 금지/지양
 - ☐ 주석 작성 (복잡한 로직)
 - ☐ Import 와일드카드 금지
 - ☐ 에러 처리 구현
 - ☐ 제네릭 타입 명시
 - ☐ 메서드명 명확하게 작성
 - ☐ Impl 사용 금지
 - ☐ `CommonResponse<T>` 구조 적용
 - ☐ 로깅 추가 (`@Slf4j`)
 - ☐ HTTP 파일 공유
 - ☐ 테스트 코드 작성
-